LocalStock

Project guide: start / stop, GitHub status, improvements, and career-readiness review

Repository: github.com/vjat5452/Master-App | Prepared 3 Nov 2026

1. What the project is

LocalStock is a hyperlocal commerce platform. Local sellers list products and stock across several warehouses; shoppers find what is available near them, compare prices, and check out with stock reserved atomically. It is a Node.js modular monolith (Express API plus a background worker) on PostgreSQL/PostGIS, Redis and RabbitMQ, with a React + Vite + Tailwind web app. Size: about 20,000 lines of JavaScript, 15 database migrations, 28 test files, 2 GitHub Actions workflows (CI and a keep-awake ping).

Piece	Where	Role
API	apps/api	REST API: auth, catalog, search, checkout, payments, admin
Worker	apps/worker	Outbox publisher, RabbitMQ consumers, reservation expiry, reconciliation, price collection
Web	apps/web	Shop, cart/checkout, orders, seller hub, admin console, API explorer
Packages	packages/*	database, allocation-engine, connectors, external-data, payments, logger, metrics
Infra	docker-compose.yml	PostgreSQL + PostGIS, Redis, RabbitMQ

2. How to start

One-time setup (already done on this machine)

```
git clone https://github.com/vjat5452/Master-App.git
cd Master-App
Copy-Item .env.example .env
npm install
```

Every time you start work

- **1.** Start **Docker Desktop** and wait until it says it is running (required after every reboot).
- **2.** Open PowerShell in `C:\p_w`. On this machine git and docker are not on PATH, so first run:

```
$env:Path = "C:\Users\User\tools\git\cmd;C:\Program Files\Docker\Docker\resources\bin;" + $env:Pat
```

- **3.** Start the infrastructure and prepare the database:

```
npm run infra:up        # PostgreSQL, Redis, RabbitMQ containers
npm run db:migrate      # first time, or after pulling new migrations
npm run db:seed         # first time only: demo sellers, warehouses, 500 products
```

- **4.** Start the three apps, each in its own terminal:

```
npm run dev:api         # http://localhost:5000   (health: /health/ready)
npm run dev:worker
npm run dev:web         # http://localhost:5173
```

- **5.** Open http://localhost:5173 and sign in (password for all: `localstock-dev-password`).

Email	Role
customer@localstock.dev	Customer

Email	Role
seller.a@localstock.dev	Seller (FreshMart Noida)
admin@localstock.dev	Platform admin

- RabbitMQ management UI: http://localhost:15672. End-to-end smoke test: `.\scripts\demo.ps1` (each run reserves 4 units of bread, so restock via an inventory adjustment after many runs).
- The repeated "Restarting 'src/server.js'" lines from `node --watch` at startup are harmless.
- Tests: `npm test` (unit/HTTP) and `$env:RUN_DB_TESTS='1'; npm run test:db` (needs the database up).

3. How to stop

- Press **Ctrl+C** in the API, worker and web terminals.
- `npm run infra:down` stops and removes the containers. **Your data is kept** in the Docker volumes.
- `docker compose down -v` additionally **deletes all data**. Use it only for a clean reset, then migrate and seed again.
- Quitting Docker Desktop also stops the containers. Other containers on this machine (for example visionai-postgres-1) are unrelated; do not stop them for this project.

4. How far it is on GitHub

Item	State (checked 3 Nov 2026)
Remote	origin = github.com/vjat5452/Master-App
main	Up to date at 3d587d2 ("Merge feature/razorpay-payments: Vercel Speed Insights"). Everything is merged.
feature/razorpay-payments	Pushed, equal to origin; the working tree is clean (nothing uncommitted). It is 7 merge/commit steps behind main, so work from main next.
feature/unified-external-platform-discovery	Pushed and fully merged into main (0 commits not in main). Can be deleted.
Latest work	Razorpay test-mode payments; stuck collection jobs fix; full API explorer; developer reference docs; Vercel Web Analytics and Speed Insights; race-condition fixes; README rewrite; architecture document and ADR.
CI	GitHub Actions ci.yml runs on push; it was green when last checked in earlier sessions (not re-checked today).
Not in git	This PDF (docs/LocalStock_Guide.pdf) is a new untracked file; it is saved but not committed. The pass/ folder is intentionally left alone.

Deployment state

- A free hosting plan exists and is documented in docs/DEPLOY_FREE.md: website on Vercel, API+worker in one process on Render, Neon (Postgres), CloudAMQP (RabbitMQ).
- Vercel Analytics and Speed Insights are in the code, so the website was deployed on Vercel at some point; the public URL is not recorded in the repo.
- From my earlier notes, the Render Docker image had not been built and tested locally. Please verify the Render backend is up before showing it to anyone.
- Render's free tier sleeps after 15 minutes; the keep-awake workflow pings it only 09:30-18:30 IST to protect Neon's free compute hours.

- Razorpay is in **test mode only**; live keys are deliberately refused. External platforms (Amazon, Flipkart, Blinkit...) are not scraped: only search links and authorized-API hooks exist.

5. What works today (from docs/PROJECT_STATUS.md)

Area	Status
Auth and tenancy	Done: scrypt passwords, JWT + rotating refresh tokens with reuse detection, company roles
Catalog, sellers, inventory	Done: variants, matching and review queue, PostGIS warehouses, audited stock ledger
Search / discovery	Done: full-text + trigram, service-area filter, Redis cache
Allocation engine	Done: splits a cart across warehouses and delivery providers, tests and micro-benchmark
Checkout	Done: Idempotency-Key, atomic multi-row reservations, orders, shipments
Payments	Done in test mode: Razorpay, signed webhooks, sync job, auto-refund of late payments, sandbox mode
Reliability	Done: transactional outbox/inbox, publisher confirms, retries, dead-letter queue, expiry, reconciliation
Delivery	Partial: quoting and assignment only; no live tracking events
Observability	Prometheus metrics and runbook done; Grafana dashboards not built; k6 script exists
External price comparison	Framework, matching, price history built; no real platform connected (needs authorized access)

6. Changes that would enhance the system

A. Close the known gaps (highest value for the effort)

- **Publish real performance numbers.** Run the existing k6 script against checkout and search, record p50/p95/p99 latency and throughput, and put the table and a short "how I fixed the bottleneck" story in the README. This is the single biggest credibility gain.
- **Prove the no-overselling claim.** Add a test that fires, say, 200 concurrent checkouts at 50 units of stock and asserts exactly 50 succeed. Show the result.
- **Shipment lifecycle:** pickup, tracking webhooks, CONSUMED reservations and DISPATCH movements (open gap #1).
- **Grafana dashboards and alert rules** from the Prometheus metrics you already expose (gap #3).
- **Refresh tokens in HttpOnly cookies** instead of browser storage (gap #5), plus CSRF protection.
- **Polygon service areas** instead of a radius (PostGIS already supports it, gap #4).

B. Engineering quality

- **Migrate to TypeScript**, at least the packages (database, allocation-engine, payments). Most large employers expect it.
- **Browser end-to-end tests** with Playwright for browse, cart, checkout and pay; run them in CI.
- **Coverage reporting** and a minimum threshold in CI; add load/regression test as a manual workflow.
- **OpenTelemetry tracing** across API, outbox, RabbitMQ and worker so one order can be followed end to end.

- **Security pass:** rate limits on auth, dependency audit in CI, secrets scanning, input-validation review, OWASP checklist in docs.
- More ADRs (only one exists): why a monolith, why outbox, why PostGIS, why RabbitMQ over Kafka.

C. Scale and architecture (to be able to discuss big-company problems)

- Move search to OpenSearch/Elasticsearch with ranking and typo tolerance; keep Postgres as source of truth.
- Read replica and connection pooling; document what breaks first at 10x and 100x traffic.
- Cache stampede protection and cache invalidation strategy; CDN for the web app.
- Extract the worker's price collection or notification path as a separate service to show a clean service boundary, without rewriting everything into microservices.
- Infrastructure as code (Terraform) and Kubernetes manifests or a Helm chart; verify the Render Docker image builds in CI.
- Recommendation or demand-forecast feature using order history, only if you can explain it well.

D. Product polish

- A short screen-recorded demo (2 to 3 minutes) and screenshots at the top of the README.
- A seller onboarding flow, order notifications by email/SMS (sandbox), and a mobile-first pass on the web app.
- Delete merged branches and keep one clean main with a tidy commit history.

7. Is this good enough for an SDE job at Amazon, Flipkart, Meta or Google?

Short answer: it is a strong project for a fresher or early-career candidate, and clearly better than the typical to-do app or e-commerce clone. By itself it will not get you an offer at these companies, because their hiring is decided mainly by data-structures-and-algorithms interviews and system-design rounds, not by the project. Treat it as a very good conversation piece that gets you past resume screens and gives you material for system design rounds.

What is genuinely impressive

- Real distributed-systems patterns, not just CRUD: transactional outbox and inbox, idempotent consumers, retries with a dead-letter queue, publisher confirms.
- Correctness under concurrency: atomic stock reservation, Idempotency-Key checkout, and fixes for checkout/session race conditions.
- Non-trivial domain logic: a multi-warehouse allocation engine with scoring, and PostGIS service-area search.
- Production concerns: CI, Docker, Prometheus metrics, a runbook, ADRs, OpenAPI spec, a payment integration with signed webhooks and refunds.
- A deployed demo and sensible ethical boundaries (no scraping of platforms that forbid it), which interviewers like to hear.
- Documentation is far above average.

What a big-company interviewer will probe or discount

- **No evidence of scale.** There are no users and no published load numbers. "Handles X orders/sec with p99 under Y ms" is worth more than ten more features.
- **Ownership questions.** They will ask why you chose each approach, what the alternatives were and how it fails. If a large part was produced with AI assistance, you must be able to explain and re-derive every important piece (the reservation SQL, the outbox flow, the allocation scoring) without notes. This is the biggest risk, and fully within your control.
- Plain JavaScript, a single modular monolith, and one ADR. TypeScript and a clearer scaling story would help.
- Payments are test mode only; delivery tracking and Grafana dashboards are unfinished; the external-comparison feature has no live data source.
- Test depth: 28 test files for about 20,000 lines; no browser E2E tests or coverage numbers.

Company by company

Company	Fit	What matters most
Flipkart	Very relevant: e-commerce, inventory, search, checkout, delivery	Flipkart interviews cover DSA plus machine-coding and low-level design. Be ready to extend the allocation or reservation logic live and discuss inventory consistency.
Amazon	Relevant: order pipeline, idempotency, reliability	Leadership Principles dominate. Prepare 6 to 8 stories from this project (ownership, dive deep, bias for action, deliver results). DSA plus system design follow.
Google	Helpful but secondary	Almost entirely DSA/algorithms plus system design. The project matters little unless it shows depth you can discuss; competitive-programming style practice matters far more.
Meta	Helpful but secondary	Fast, coding-heavy rounds (LeetCode medium/hard) plus system design. Product and speed count; the project is mostly a talking point.

What to do next to maximise your chances

- **Spend the majority of your time on DSA** (arrays, hashing, trees, graphs, DP, heaps; 250 to 400 well-understood problems) and on system-design practice. This decides these interviews.
- **Add the benchmark and concurrency-proof results** (section 6A), then write 3 resume bullets with numbers, for example: "Built an inventory reservation system with atomic multi-row stock holds; verified zero overselling under 200 concurrent checkouts; p95 checkout latency Y ms."
- **Prepare a 10-minute walkthrough**: the problem, architecture diagram, one hard bug (the race conditions), one trade-off (monolith vs microservices, outbox vs direct publish), and what you would change at 100x scale.
- **Get a referral** or apply through campus/off-campus drives; a good project helps a resume get read, but referrals move it to the front.
- Also target strong product companies and well-funded startups (Swiggy, Zomato, Razorpay, Atlassian, Uber, PhonePe...). This project fits them well and they are realistic stepping stones.
- Keep an honest framing: say clearly what is test-mode, demo or unfinished. Interviewers trust candidates who know their system's limits.

Bottom line: project quality is a good 7 to 8 out of 10 for a fresher portfolio. For Amazon and Flipkart it is a solid supporting asset; for Google and Meta it will not carry you, and the DSA rounds will. With benchmarks, TypeScript and the ability to explain every design choice from memory, it becomes a standout rather than just a good project.